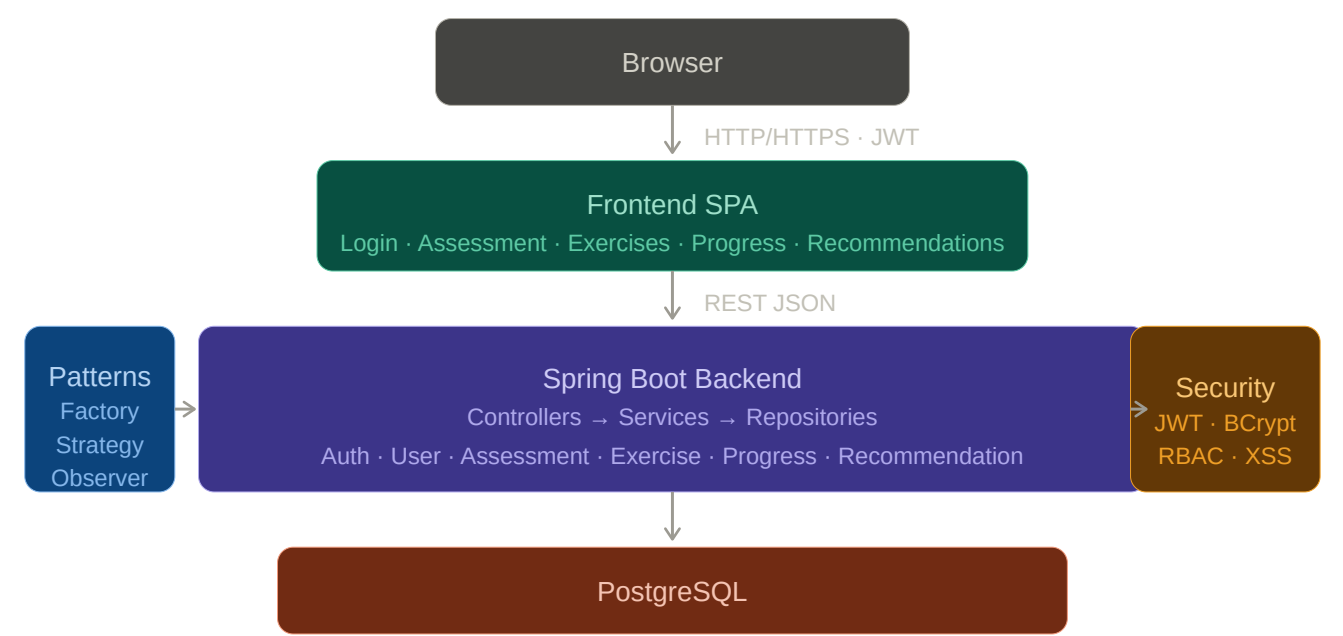


SocialEase: System Architecture Design Document

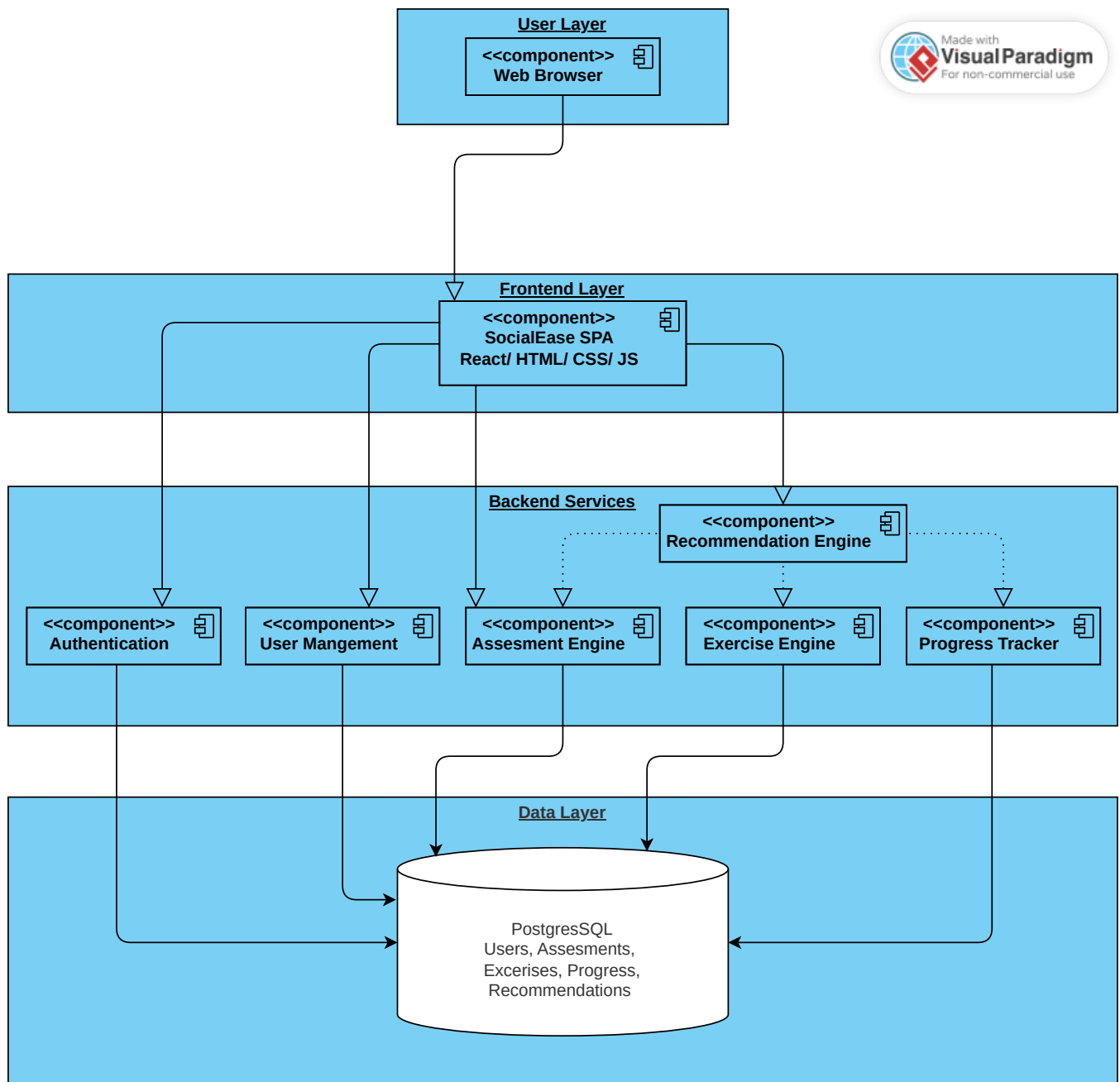
1. Brief System Overview

SocialEase is an interactive web application designed to help individuals overcome social anxiety through personalized exercises, anxiety assessments, and progress tracking. The system consists of five main modules: User Management (registration, login, profile), Assessment Module (anxiety level evaluation), Exercise Module (interactive tasks and challenges), Progress Tracker (improvement monitoring), and Recommendation System (personalized activity suggestions). It follows a layered MVC architecture with Spring Boot on the backend and PostgreSQL for data persistence.

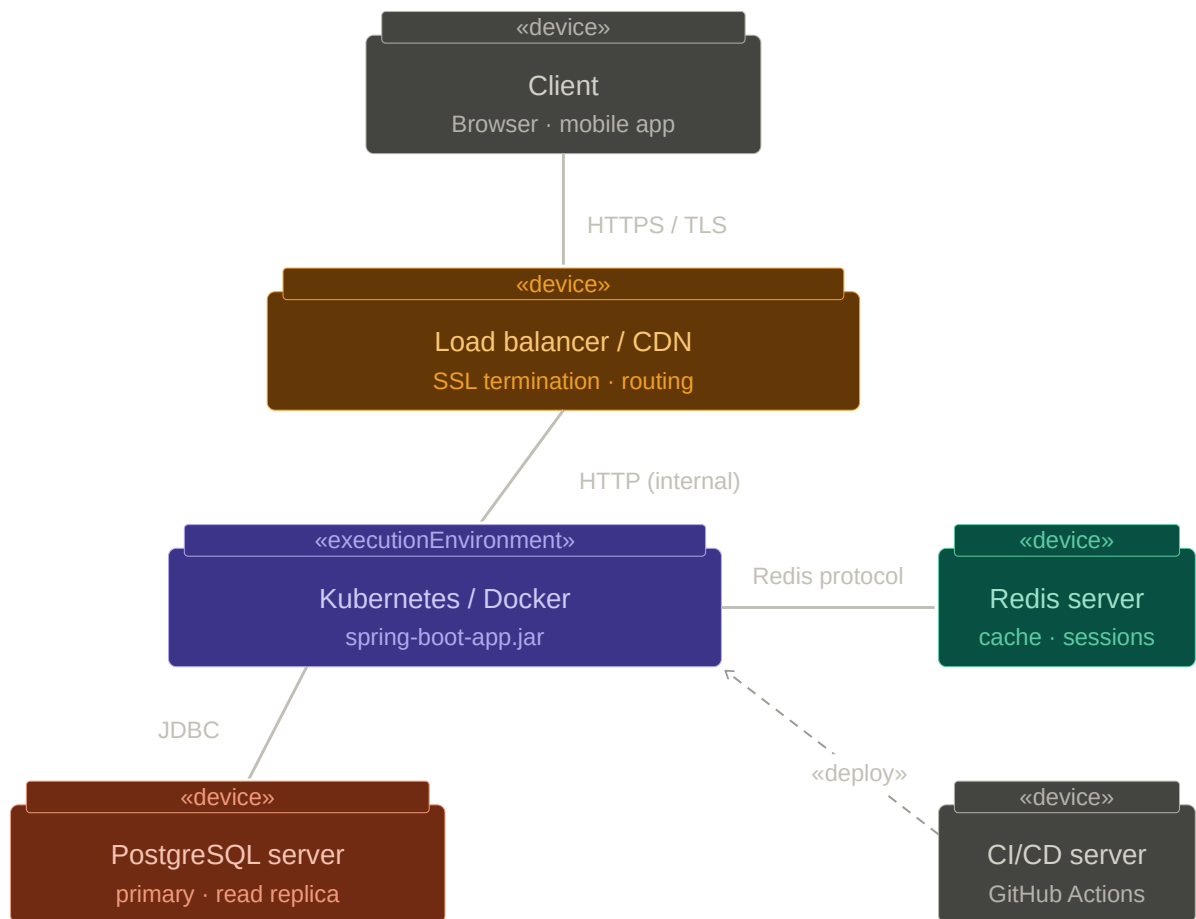
2. High Level Architecture Diagram



3. Component Diagram



4. Deployment Diagram



5. Data Flow Overview

1. User Registration

User fills out signup form → Frontend sends data to backend → Backend checks if email exists → Hashes password → Saves user to database → Returns success with login token → User stores token and is ready to login.

2. User Login

User enters email and password → Frontend sends to backend → Backend looks up user in database → Compares password → If match, generates login token → Returns token → User stores it and is logged in.

3. Taking an Assessment

User answers 10-15 anxiety questions → Clicks submit → Frontend sends answers to backend → Backend calculates anxiety score → Saves assessment to database → Records progress metric → Sends back results → Frontend shows the user their anxiety level.

4. Browsing Exercises

User picks an exercise → Frontend notifies backend → Backend marks as 'started' in database → Frontend updates UI to show the exercise is active.

5. Getting Recommendations

System looks at user's anxiety level and past exercises → Uses recommendation algorithm → Picks exercises that match their needs → Saves recommendations to database → Frontend displays them → User can accept or dismiss.

6. Viewing Progress

User clicks "My Progress" → Frontend asks backend for stats → Backend calculates: total exercises done, average score, current streak, anxiety trends → Queries database for all user's progress data → Returns numbers → Frontend displays charts and stats.

6. Final Selected Design Patterns

Pattern 1: Factory Pattern

Property	Description
Category	Creational
Where Used	Exercise creation module
Why Suitable	Instead of writing code to create each exercise type separately, the Factory pattern creates any exercise type from one place just ask for what you need and it builds it.
Expected Benefit	Adding new exercise types requires only adding a new concrete factory class no modification to existing exercise creation code. This makes the system extensible and maintainable.
Related Classes	<code>BreathingExerciseFactory</code> , <code>ConversationExerciseFactory</code> , <code>PublicSpeakingExerciseFactory</code> , <code>ExerciseCreator</code>

Pattern 2: Strategy Pattern

Property	Description
Category	Behavioral
Where Used	Recommendation system
Why Suitable	The system chooses different recommendation strategies based on what the user needs beginners get easy exercises, advanced users get challenging ones.
Expected Benefit	The recommendation logic is isolated into separate strategy classes, making it easy to add, modify, or swap algorithms without affecting the rest of the system. This supports personalized user experiences.
Related Classes	<code>AnxietyBasedStrategy</code> , <code>ProgressBasedStrategy</code> , <code>CombinedStrategy</code> , <code>RecommendationService</code>

Pattern 3: Singleton Pattern

Property	Description
Category	Creational
Where Used	Database connection management and application configuration
Why Suitable	The app uses one database connection that never gets duplicated Singleton makes sure there's only one and everyone uses the same one.
Expected Benefit	Centralized control over shared resources, reduced memory footprint, consistent configuration state across the application, and prevention of connection pool conflicts.
Related Classes	<code>DatabaseConnection</code> , <code>AppConfig</code>

7. Architecture Justification

Why Layered MVC Architecture is Suitable

Suitability for SocialEase: The layered MVC architecture is well-suited for SocialEase because it separates concerns across clear boundaries presentation (controllers), business logic (services), and data access (repositories). This separation maps naturally to the needs of a web application with multiple modules that share the same infrastructure but have distinct responsibilities.

Maintainability

Each layer has a single responsibility and can be modified independently:

- Frontend changes (UI redesign) do not affect backend logic
- Business rule changes only affect the service layer

Scalability

The architecture supports horizontal scaling:

- The frontend (SPA) can be served via CDN for fast global access
- The database can be scaled vertically or through read replicas

Reusability

Components and services are designed as reusable units:

- Progress metrics logic can be reused in both the progress summary and trend analysis features
- DTOs and utility classes are shared across related controllers

Design Pattern Integration

The layered architecture provides natural integration points for design patterns:

- **Factory Pattern** fits naturally in the service layer where exercise object creation occurs
- **Strategy Pattern** integrates cleanly into the recommendation service where algorithms vary
- **Singleton Pattern** aligns with infrastructure-level concerns like database connections

8. Team Contribution

Section	Contributor
1. Brief System Overview	Miral Adel
2. High-Level Architecture Diagram	Miral Adel
3. Component Diagram	Rana Mostafa
4. Deployment Diagram	Snaa Elsayy
5. Data Flow Overview	Snaa Elsayy
6. Final Selected Design Patterns	Snaa Elsayy
7. Architecture Justification	Rana Mostafa